

Ouvrir un nouveau foyer

Mode opératoire technique. Version 1.0 — 2026-08-24. Destiné à la personne qui installe l'application, pas au foyer client. Compter **45 minutes**, dont une bonne moitié d'attente (DNS, emails).

L'application sert plusieurs foyers depuis **un seul déploiement**. Chaque foyer a sa **propre base Supabase** et sa **propre adresse** ; c'est le nom d'hôte de la requête qui détermine la base atteinte. Aucune donnée ne circule entre foyers.

Ouvrir un foyer, c'est donc : créer sa base, la monter, l'inscrire au registre, et passer la main à quelqu'un chez le client.

0. Avant de commencer

Accès nécessaires

- le dépôt cloné en local, avec `node` et `npm` ;
- un compte Supabase pouvant créer un projet ;
- l'accès au projet Vercel qui héberge l'application ;
- les identifiants SMTP (Brevo ou autre) — les mêmes que les autres foyers.

À demander au foyer

Information	Sert à
Nom complet du foyer	onglet du navigateur, emails
Nom court (2 mots max)	sous l'icône sur un téléphone
Adresse email de la personne qui administrera	invitation de super-administratrice
Logo	en-tête des écrans — facultatif, le nom s'affiche à défaut

Le reste — blocs, étages, chambres, options de repas, groupes — est **saisi par le foyer lui-même**. Ne le faites pas à sa place : cette saisie est le paramétrage.

Choisir un identifiant court (le *slug*), en minuscules sans accent : `guerledan`, `bellevue` ... Il servira au registre et au sous-domaine.

1. Créer la base

Sur supabase.com → **New project**.

- **Name** : le nom du foyer ;

- **Region** : la plus proche du foyer (eu-west-3 pour la France) ;
- **Database password** : engendrez-le et **conservez-le** — il ne se réaffiche pas.

Attendre la fin de la création (1 à 2 minutes), puis relever dans **Settings → API** :

- **Project URL** — https://xxxx.supabase.co
- anon public
- **service_role** ⚠ **secrète** : jamais dans le dépôt, jamais dans une variable préfixée NEXT_PUBLIC_.

Créer un fichier .env.<slug> à la racine du dépôt (le .gitignore couvre .env*) :

```
NEXT_PUBLIC_SUPABASE_URL=https://xxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ...
```

2. Monter le schéma

Dans **SQL Editor** du nouveau projet, coller et exécuter **dans cet ordre**, un fichier à la fois :

#	Fichier	Contenu
1	supabase/migrations/20260824120000_socle.sql	22 tables, 49 policies, RLS partout, 4 fonctions, index
2	supabase/p4-options-evenement-en-code.sql	retire deux tables devenues inutiles
3	supabase/storage-branding.sql	dépôt du logo et de l'icône
4	supabase/seed.sql	contenu d'un foyer vierge

La règle générale : prendre le socle le plus récent de supabase/migrations/ , puis passer **tous** les fichiers .sql restés à la racine de supabase/ dans l'ordre alphabétique, entre le socle et le seed — hors audit-rls.sql, verif-socle.sql et sync-modes-emploi-inapp.sql , qui sont des outils et non des migrations. Une régénération du socle les absorbe et vide cette liste.

Un foyer neuf peut sauter p4 sans dommage — il ne fait que supprimer deux tables inutilisées — mais le passer garde les foyers strictement identiques, ce qui est la condition pour qu'une régénération de socle fasse foi.

Pourquoi trois fichiers et pas un. pg_dump --schema=public ne capture que le schéma applicatif : ni le bucket de fichiers, qui vit dans le schéma storage , ni les données. Un socle régénéré ne contiendra donc **jamais** le dépôt d'images — et son absence ne se signale par aucune erreur, seulement par un téléversement de logo qui échoue.

Maintenance : régénérer le socle. Quand la liste des migrations postérieures s'allonge, repartir d'un foyer à jour :

```
pg_dump --schema-only --schema=public --no-owner "$PGURL" > supabase/migrations/<date>
```

`$PGURL` est la chaîne **Session pooler (port 5432)** de *Settings* → *Database* → *Connection string* → *URI* — pas le port 5432, que `pg_dump` ne supporte pas. Trois retouches obligatoires sur la sortie :

1. retirer les méta-commandes `\restrict / \unrestrict` (`pg_dump 18`) ;
2. passer `CREATE SCHEMA public` en `CREATE SCHEMA IF NOT EXISTS public` ;
3. **supprimer le bloc `ALTER DEFAULT PRIVILEGES` de fin** — l'éditeur SQL de Supabase le refuse (permission denied to change default privileges), et ces lignes sont de toute façon redondantes sur un projet neuf.

Il faut des outils client PostgreSQL **au moins aussi récents que le serveur** : `brew install libpq`, puis `/opt/homebrew/opt/libpq/bin/pg_dump`. Une fois le nouveau socle en place, déplacer les migrations qu'il absorbe dans `supabase/migrations/archive/`.

Vérifier — coller `supabase/verif-socle.sql`. Attendu :

```
tables 22 · policies 49 · tables_rls 22 · fonctions 4 · triggers 1  
reglages_seed 10 · rubriques_seed 4
```

3. Créer le compte technique

C'est **votre** compte d'installation : tous les droits, aucune chambre, jamais compté dans la capacité du foyer, jamais listé nulle part.

```
FOYER_URL='https://xxxx.supabase.co' \  
FOYER_SERVICE_KEY='eyJ...' \  
FOYER_ADMIN_EMAIL='votre.adresse@exemple.fr' \  
node scripts/foyer-nouveau.mjs
```

Le script vérifie que le socle et le seed sont en place, refuse de se rejouer, et **affiche le mot de passe une seule fois** — notez-le immédiatement, il n'est écrit nulle part.

4. Régler les emails

Sans SMTP, Supabase interdit de modifier les gabarits et n'envoie que quelques emails par heure. C'est donc obligatoire.

Project Settings → **Authentication** → **SMTP Settings**

Champ	Valeur
Host / Port / Username / Password	les mêmes que les autres foyers
Sender name	⚠ le nom de CE foyer
Sender email	une adresse validée chez le fournisseur

Le `Sender name` doit être propre au foyer : une invitée qui reçoit un email signé du nom d'un autre foyer le prendra pour de l'hameçonnage.

Authentication → Emails → Invite user : coller le contenu de `supabase/email-invitation.html`.

Le piège le plus coûteux. Le gabarit **par défaut** de Supabase pointe vers son propre `/auth/v1/verify` : il vérifie le jeton, le **consomme**, puis redirige avec un code que l'application ne sait pas traiter. L'invitée lit « Lien invalide ou expiré » alors que son lien était valide, et le jeton est brûlé. Voir `supabase/GABARITS-EMAIL.md`.

Authentication → URL Configuration — à remplir à l'étape 5, une fois l'adresse connue.

5. Publier l'adresse

Vercel → le projet → Settings → Domains → Add : ajouter `<slug>-foyer.vercel.app` (ou un domaine propre si le foyer en a un).

Puis fabriquer le registre, en listant **tous** les foyers, l'ancien compris :

```
node scripts/foyers-json.mjs \
  ecoles=.env.local.ecoles:les-ecoles.vercel.app \
  <slug>=.env.<slug>:<slug>-foyer.vercel.app
```

Le script refuse deux foyers qui partageraient une base ou un hôte — l'erreur qui servirait les données d'un foyer sous l'adresse d'un autre.

Settings → Environment Variables :

Nom	Valeur	Environnement
FOYERS	le JSON produit, sur une seule ligne	Production
FOYER_DEV	le slug d'un foyer de test	Preview

`FOYER_DEV` n'est pas optionnel : les préversions de Vercel ont un hôte absent du registre et se replieraient sinon sur le premier foyer de la liste.

Redéployer — Deployments → le dernier → ... → Redeploy. Next fige les variables du middleware au build : enregistrer ne suffit pas.

Revenir à Supabase → Authentication → URL Configuration :

- Site URL : `https://<slug>-foyer.vercel.app`
- Redirect URLs : la même, suivie de `/**`

6. Vérifier

Deux contrôles automatiques, à lancer depuis le dépôt.

Le cloisonnement des adresses

```
node scripts/verif-foyers.mjs https://les-ecoles.vercel.app https://<slug>-foyer.vercel.app
```

Attendu :  Chaque adresse sert un foyer distinct, sur sa propre base. Deux adresses qui renverraient le même nom signeraient un registre qui ne reconnaît pas un hôte et se replie.

L'étanchéité des données — ajouter temporairement dans `.env.local` :

```
VERIF_EMAIL=...  
VERIF_PASSWORD=...
```

puis `node scripts/verif-rls.mjs --ecriture`. Attendu : 0 fuite anonyme, et aucune table requise vide une fois connectée. **Retirer les deux lignes ensuite.**

7. Passer la main

Se connecter avec le compte technique, puis **Administration → Identité du foyer** :

1. **Super-administratrices** (panneau visible du seul compte technique) — saisir l'adresse de la personne qui administrera le foyer, et envoyer l'invitation.
2. Elle reçoit un email, définit son mot de passe, et arrive avec tous les droits. **Aucun logement ne lui est demandé** : au démarrage, aucune chambre n'existe.

À partir de là, elle est autonome. **Envoyez-lui docs/installer-son-foyer.md** (ou son PDF) : c'est le même parcours, raconté de son point de vue, sans technique.

En résumé, ce qu'elle fait, dans cet ordre :

	Où
Nom, nom court, description, logo, icône	Administration → Identité du foyer
Fuseau horaire et format des dates	idem — à changer seulement hors métropole
Blocs, étages, chambres, postes	Administration → Comptes & chambres

	Où
Se choisir une chambre	encadré bleu en tête d'Administration, tant qu'elle n'en a pas
Inviter l'intendance et régler ses droits	Administration → Comptes & chambres
Options de repas, heures de verrouillage	Repas → Paramétrer les repas
Rubriques de l'onglet Administratif	Administratif

Enfin, importer les modes opératoires dans l'application : `node scripts/docs/md2tiptap.mjs`, puis exécuter le SQL produit.

8. Dépannage

Symptôme	Cause probable
« Lien invalide ou expiré », compte marqué confirmé	gabarit d'email par défaut : Supabase a consommé le jeton (§4)
« Lien invalide ou expiré », compte non confirmé	<i>Site URL</i> pointe ailleurs, ou lien réellement périmé (14 jours)
L'adresse ouvre le mauvais foyer	<code>host</code> du registre ≠ domaine réel. Domaine nu, sans <code>https://</code> ni <code>/</code>
Le nom du foyer reste « Foyer »	<code>p2-identite-foyer.sql</code> non passé, ou identité non saisie
L'icône est sur fond noir sur le téléphone	icône transparente. Il en faut une carrée et opaque (§7)
Le logo ne change pas après téléversement	cache du navigateur ; sur mobile, supprimer puis recréer le raccourci
Un écran est vide alors qu'il devrait être rempli	une migration manque : rejouer <code>verif-socle.sql</code>
<code>permission denied to change default privileges</code>	bloc <code>ALTER DEFAULT PRIVILEGES</code> resté dans un socle régénéré (§2)
Une chambre libérée refuse d'être supprimée	occupante encore active, ou invitation en attente — le message le dit
Aucun type d'événement proposé, création impossible	déploiement antérieur au 2026-08-25. Les types et délais de rappel vivaient dans deux tables que le seed ne remplissait pas : un foyer neuf n'en avait aucun, et la catégorie étant obligatoire, aucun événement n'était créable. Ils sont désormais dans le code — redéployer suffit

Les journaux Vercel portent l'essentiel : `auth/confirm` y trace l'hôte, le type de jeton et le motif exact d'un refus ; le registre y signale tout hôte inconnu.

9. Récapitulatif

- ☐ Projet Supabase créé, mot de passe conservé
- ☐ `.env.<slug>` rempli, hors du dépôt
- ☐ socle + migrations restantes + `storage-branding.sql` + `seed.sql`, dans l'ordre ; `verif-socle.sql` conforme
- ☐ Compte technique créé, mot de passe noté
- ☐ SMTP réglé, **Sender name propre au foyer**
- ☐ Gabarit d'invitation collé
- ☐ Sous-domaine ajouté sur Vercel
- ☐ `F0YERS` en Production, `F0YER_DEV` en Preview, **redéployé**
- ☐ *Site URL* et *Redirect URLs* renseignées
- ☐ `verif-foyers.mjs` et `verif-rls.mjs` au vert
- ☐ Super-administratrice invitée
- ☐ Modes opératoires importés